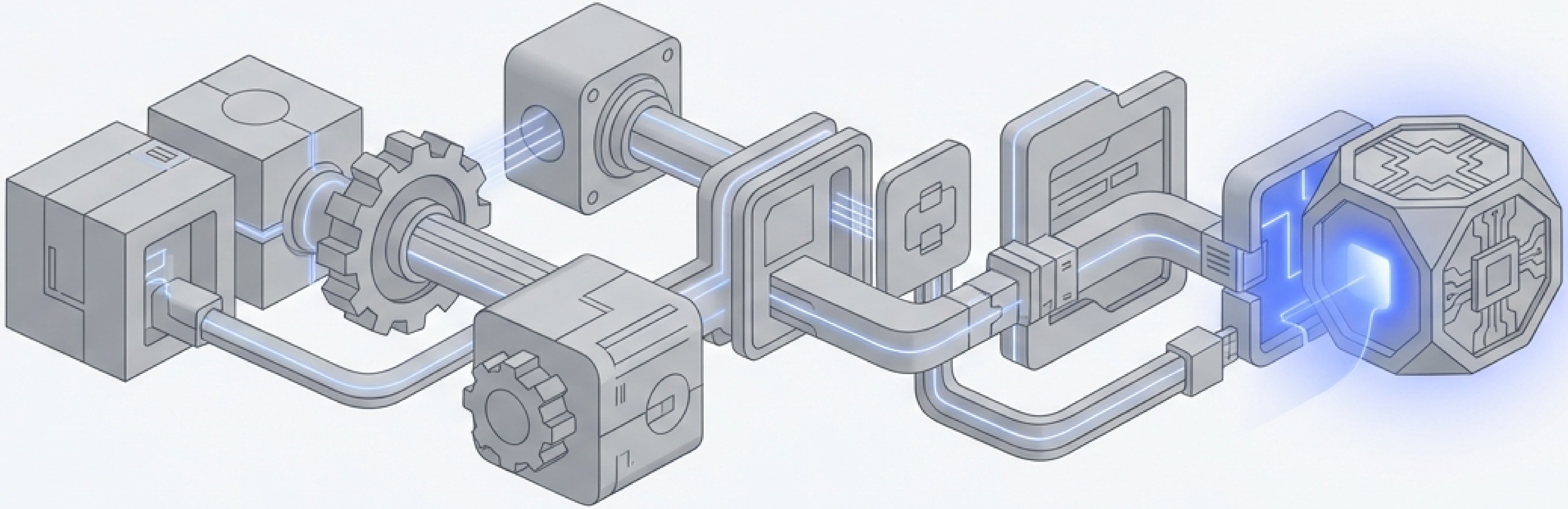


Activate Your Decentralized Exchange

A Guided Walkthrough for Live Testing on the Sepolia Testnet

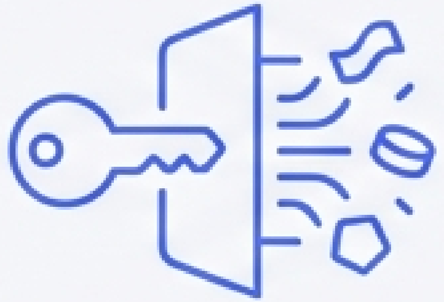
Your Mission: From Static Code to a Live Exchange

You’ve successfully coded and deployed your Token and Exchange smart contracts to the Sepolia testnet. Now it's time to activate them.

We will use Etherscan as our control panel to perform three critical user actions.

Seed the Pool: Add initial liquidity, the lifeblood of the exchange.

Execute Swaps: Test the core engine by swapping ETH for Tokens and back.

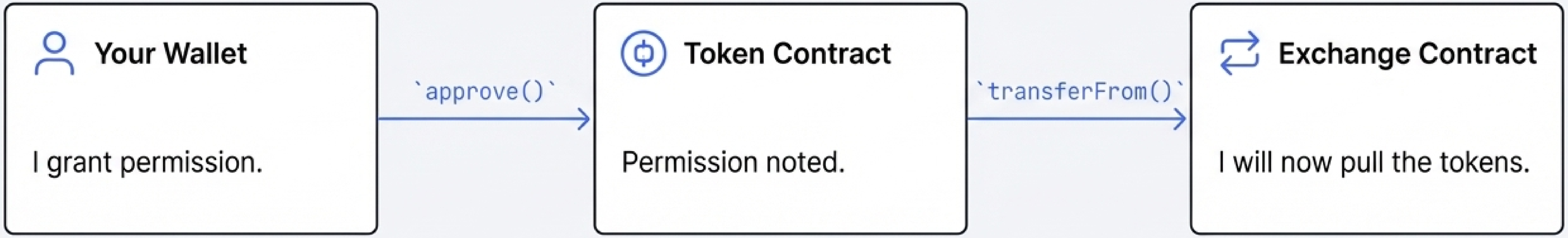


Reclaim Assets: Prove the system is trustworthy by withdrawing your liquidity.

Have your deployed `Token` and `Exchange contract addresses` on `Sepolia Etherscan` ready. You should have `1,000,000` TKN in your deployment wallet.

Action 1: Seeding the Pool

An exchange is useless without liquidity. Your first task is to provide the initial ETH and TOKEN reserves. This action relies on a fundamental ERC-20 security pattern: the `approve` and `transferFrom` flow.



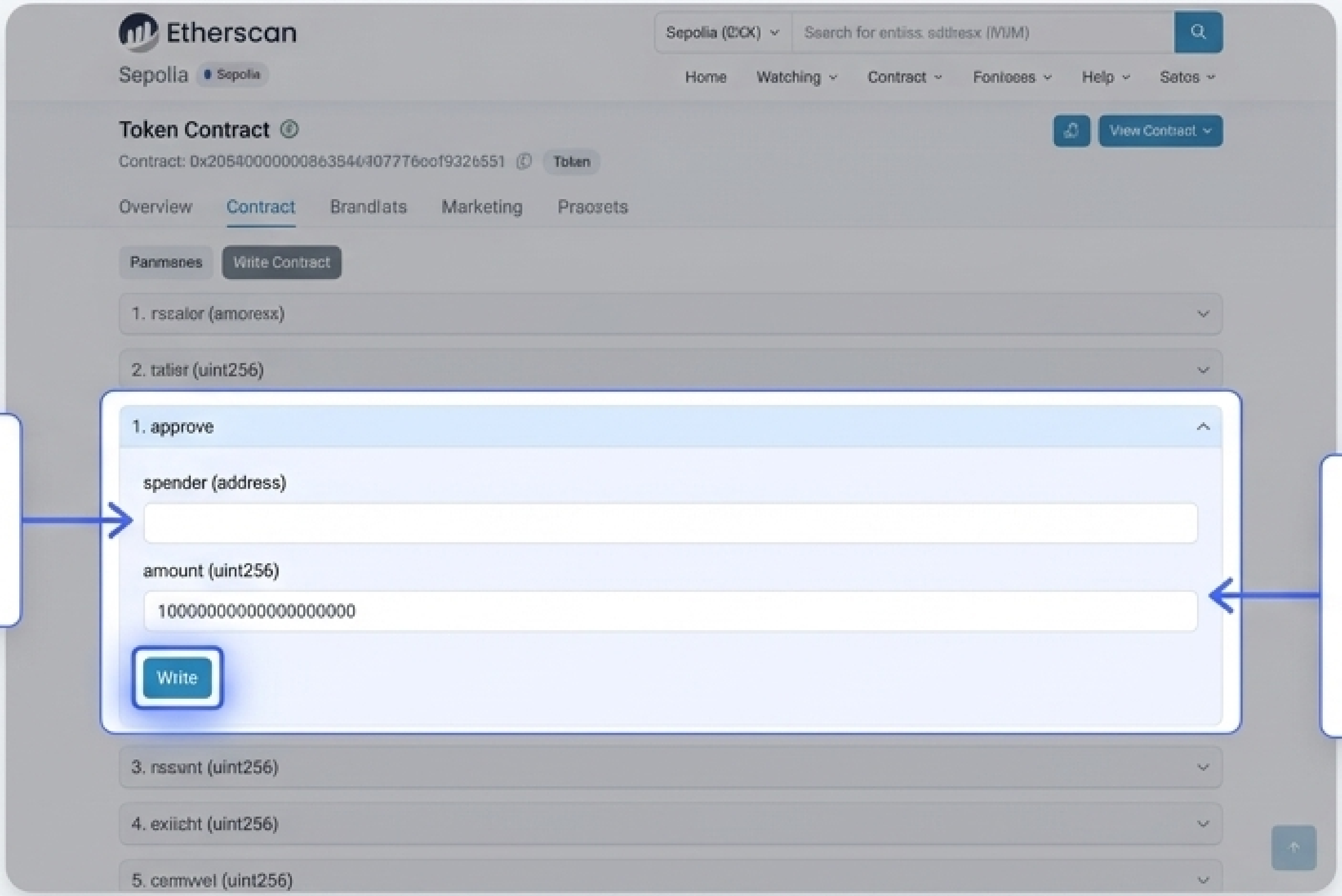
Why approve first?

You are giving the Exchange contract *permission* to pull a specific number of tokens from your wallet on your behalf. This is a core security feature of the ERC-20 standard, preventing contracts from draining your tokens without consent.

Step 1: Granting Token Allowance on Etherscan

Navigate to your **Token contract** on Sepolia Etherscan. Go to the `Contract` tab, then click `**Write Contract**` and connect your wallet.

Enter your **Exchange contract address** here. This is who you're giving permission to.

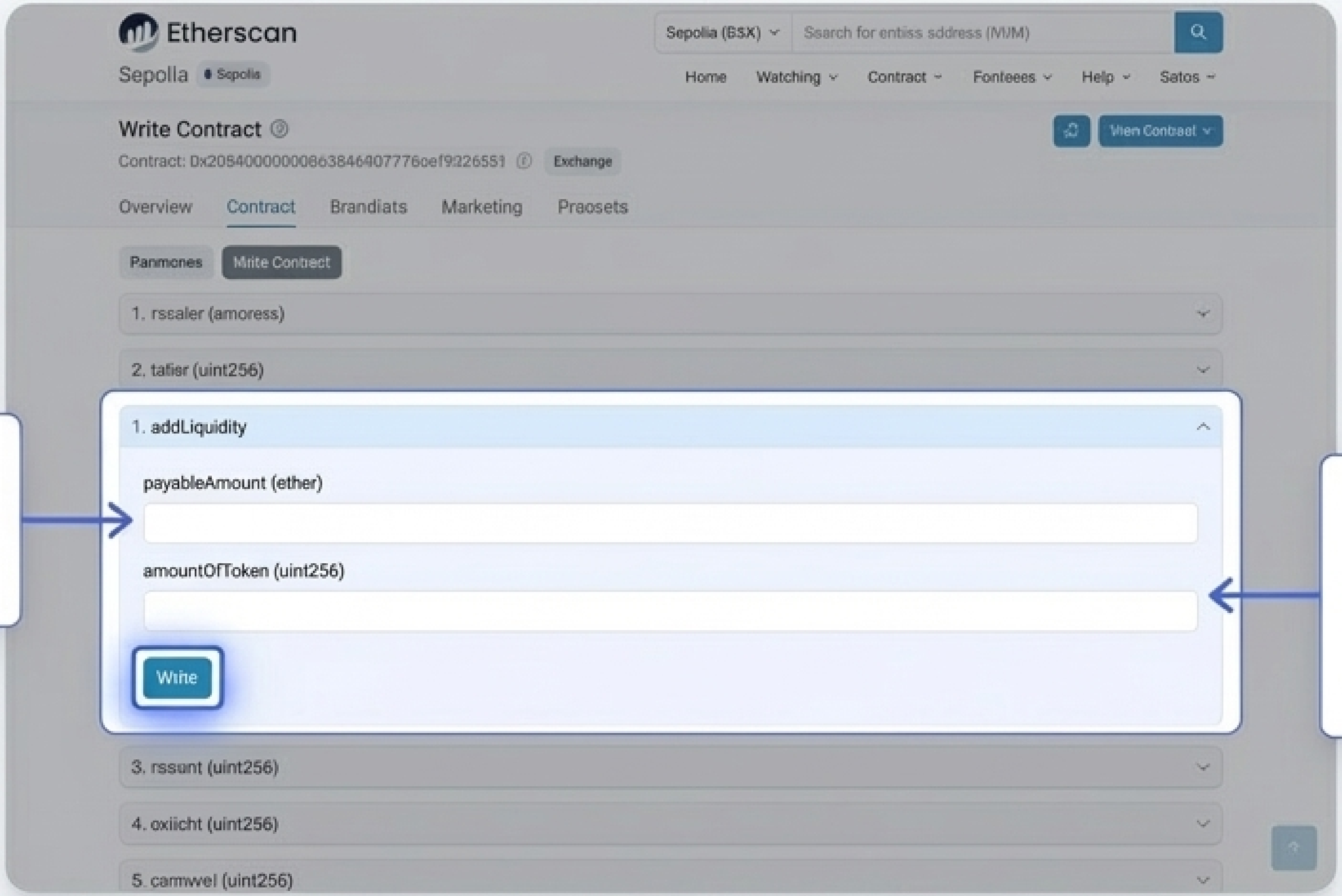


Enter the amount of tokens to approve. For this test, let's use `10000000000000000000` (which is 10 TKN, assuming 18 decimals).

After clicking 'Write', confirm the transaction in your wallet.

Step 2: Adding Liquidity to the Exchange

Now, navigate to your **Exchange contract** on Etherscan. Go to `Contract` → `Write Contract` and ensure your wallet is connected.



Enter the **amount of ETH** you want to provide. Let's use `0.1`.

Enter the **amount of TOKENs**. Let's use `10000000`. This must be less than or equal to the amount you just approved.

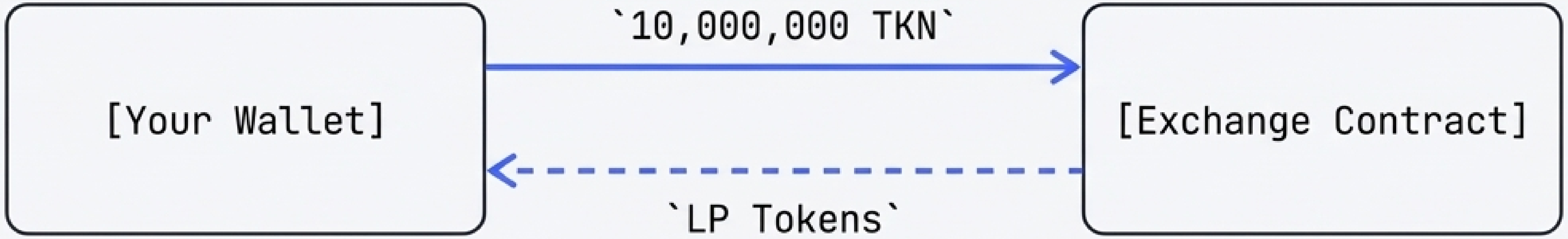
After clicking 'Write', confirm the transaction in your wallet.

NotebookLM

Success! You Are Now a Liquidity Provider.

Once your transaction is confirmed, check the transaction details on **Etherscan**. You will see two key transfers:

1. Your TKNs were transferred from your wallet to the Exchange contract.
2. New “ETH TOKEN LP” tokens were minted and transferred into your wallet.

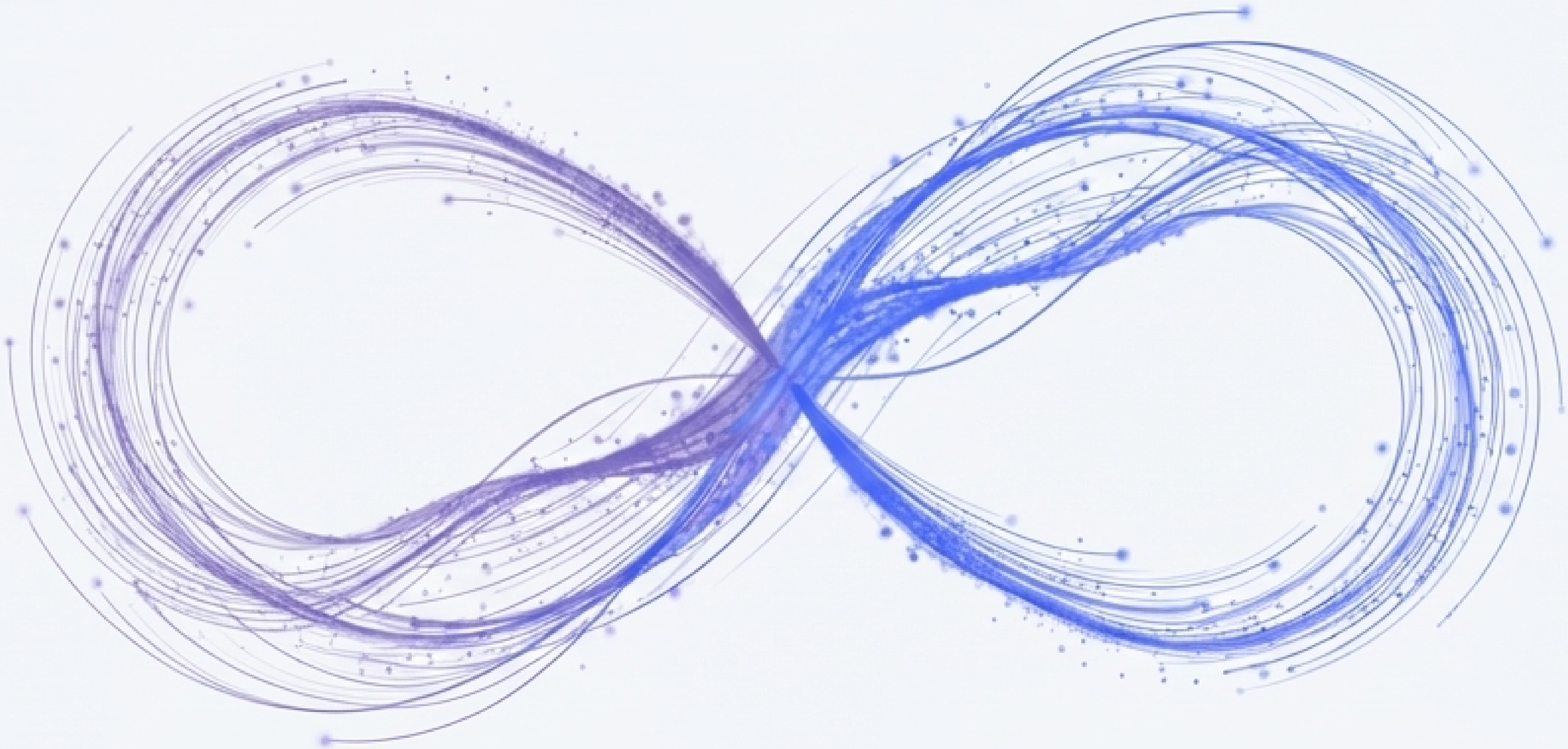


What are LP Tokens?

These tokens represent your proportional share of the liquidity pool. You can hold them to earn fees from swaps, or burn them later to reclaim your share of the underlying ETH and TKN.

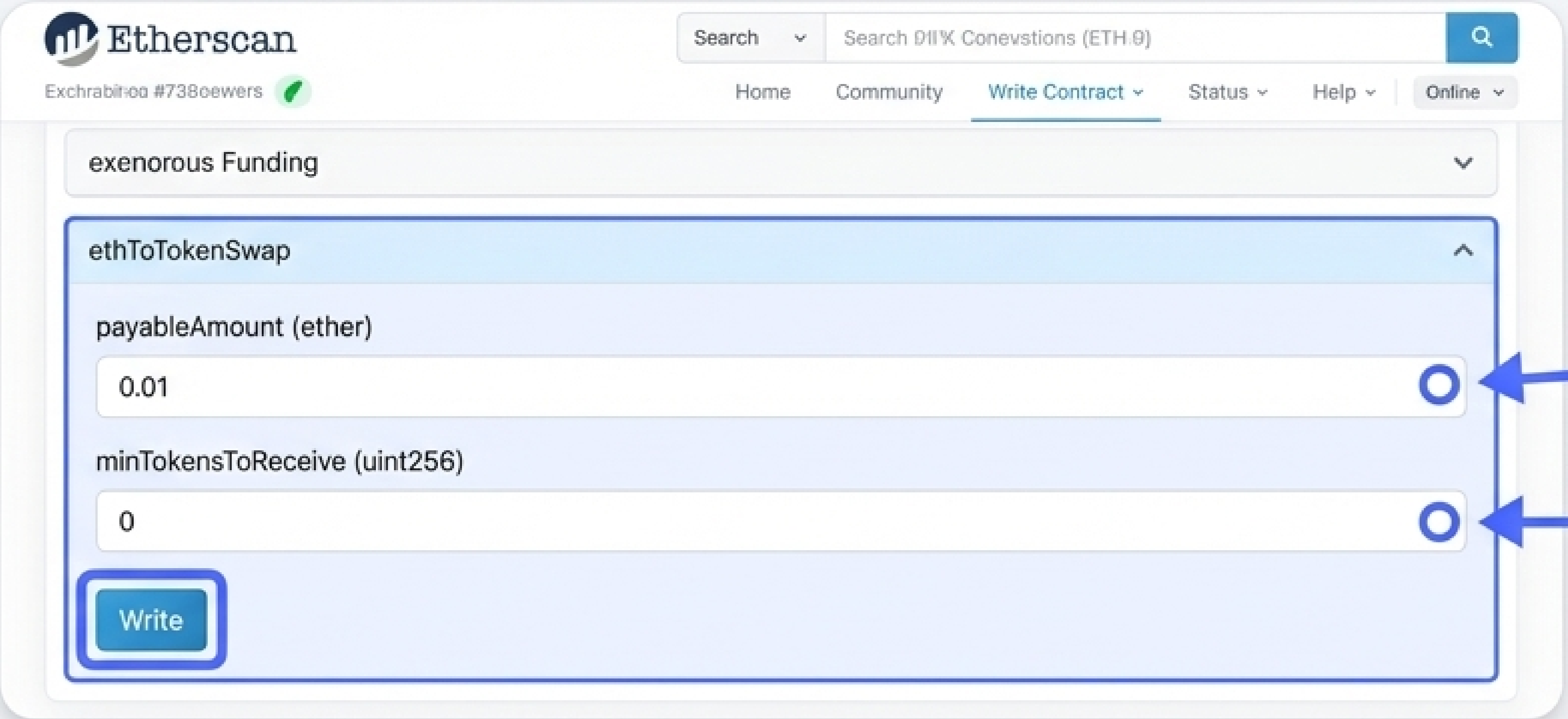
Action 2: Firing Up the Swap Engine

With liquidity in the pool, the exchange is now operational. It's time to test its primary function: swapping one asset for another. We will perform two swaps to test both directions of the ETH <> TOKEN pair.



Swap 1: Trading ETH for TOKEN

On the Exchange contract's `Write Contract` page, find the `ethToTokenSwap` function.



Enter the amount of ETH to swap. Since we added 0.1 ETH liquidity, let's use a smaller amount like `0.01`.

For this initial test, we can set this to `0`. We'll explore why this is important next.

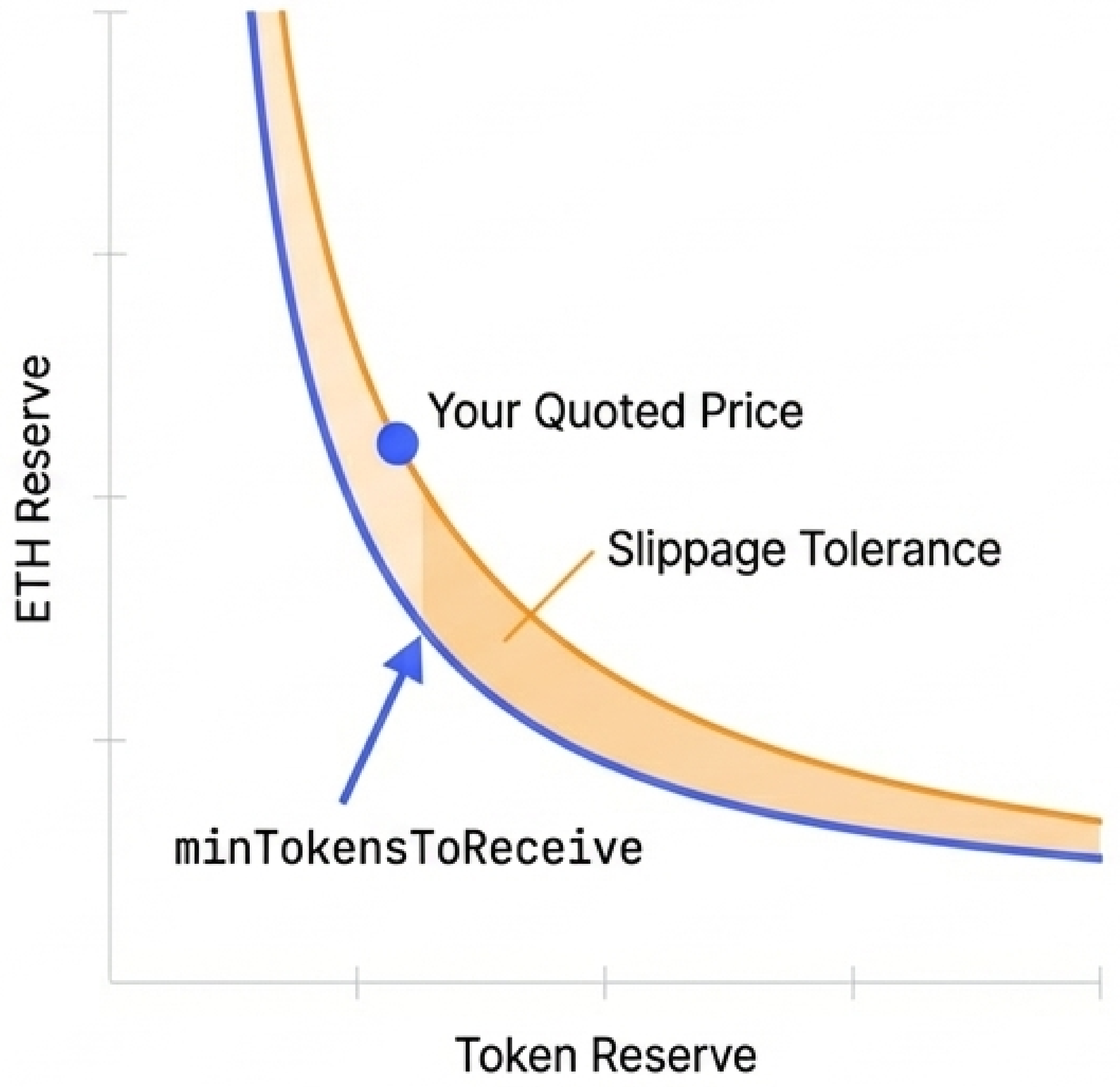
After the transaction confirms, you will see TOKENs transferred back to your wallet in exchange for your ETH.

Expert Insight: Understanding Slippage Protection

In a real-world scenario, you would never set `minTokensToReceive` to zero. Here's why:

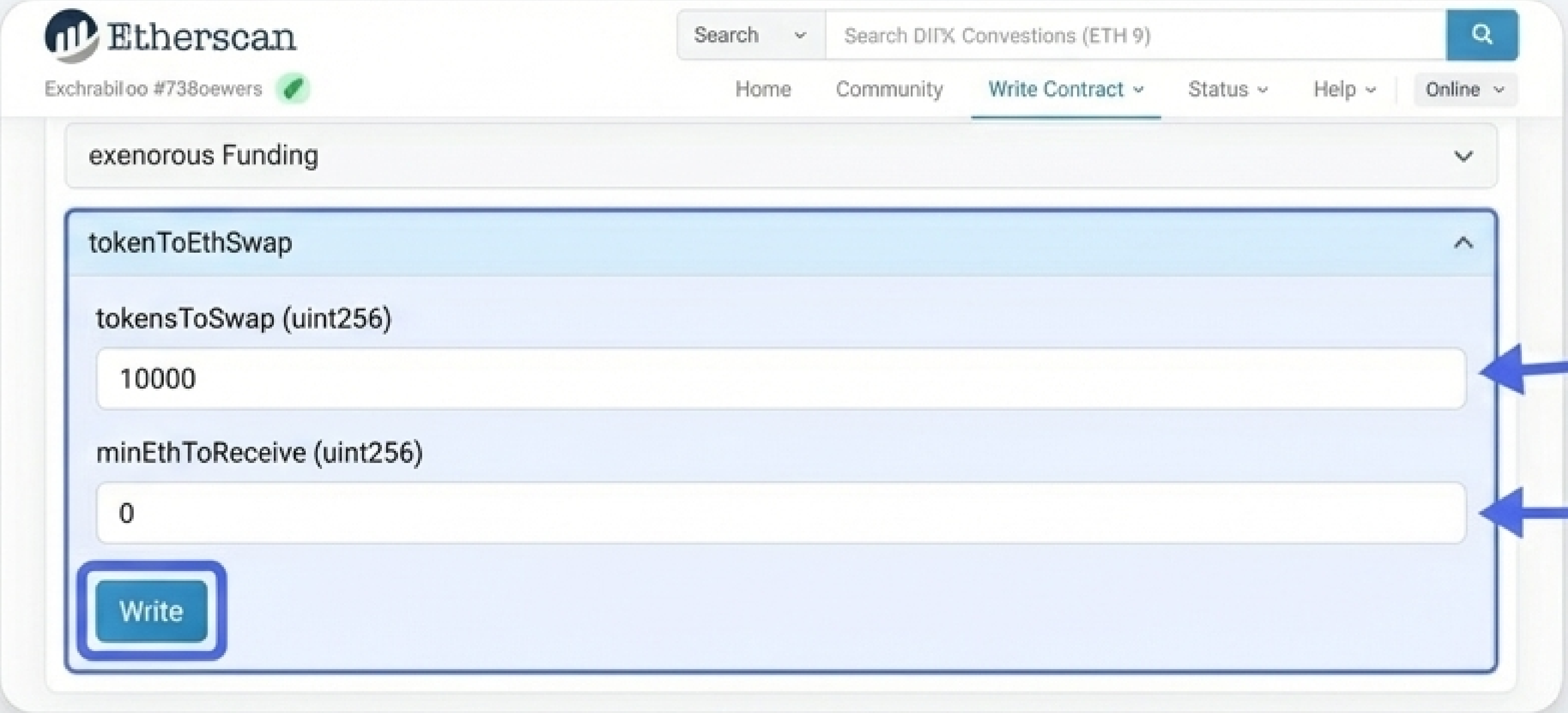
- **What is Slippage?** The price of tokens in an AMM pool changes with every trade. By the time your transaction is confirmed, the price might have shifted against you.
- **How Protection Works:** The user interface (the website or app) estimates the tokens you'll receive (e.g, 100 TKN). You agree to a slippage tolerance (e.g., 5%). The UI then calculates the minimum you're willing to accept (95 TKN) and sets this as the `minTokensToReceive` parameter.

> This protects you from unexpected price changes. If the transaction would result in fewer than 95 TKN, it automatically fails, protecting your funds.



Swap 2: Trading TOKEN for ETH

Now, find the **`tokenToEthSwap`** function. Remember you may need to **`approve`** the Exchange contract again on the Token contract if you want to swap more tokens than your remaining allowance.



Enter the amount of TOKEN to swap. Let's use `10000`.

Again, for this test, we can set this to `0`.

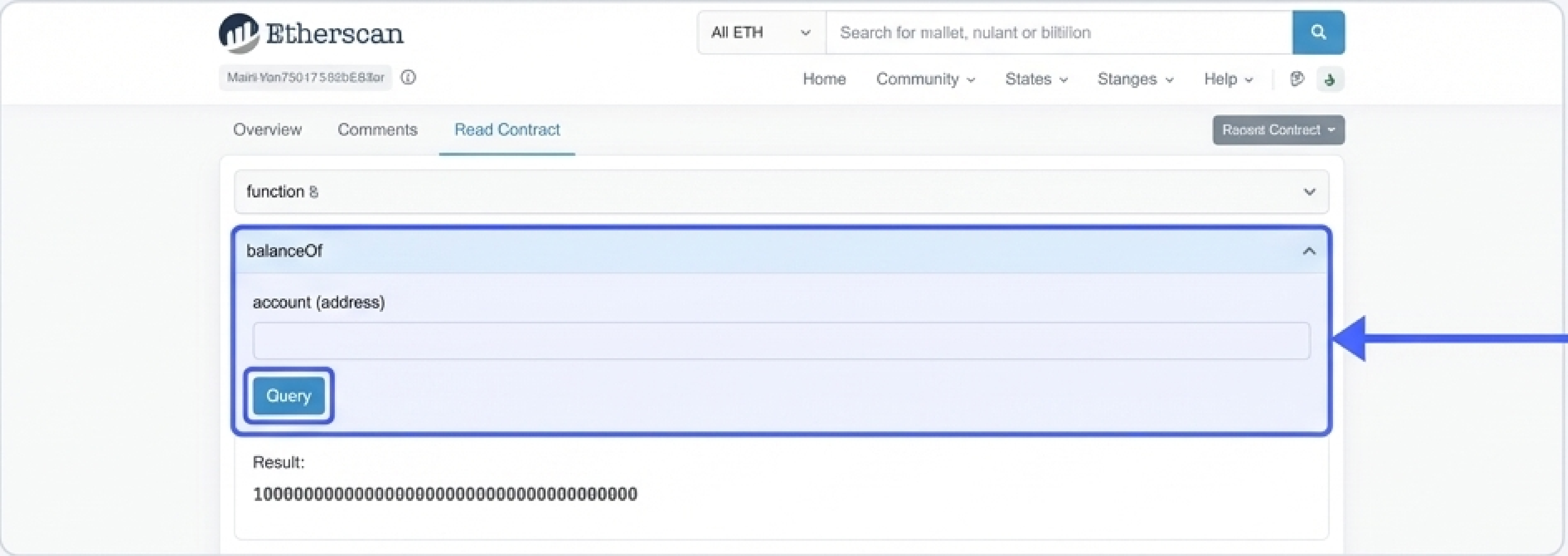
Success! You'll see ETH transferred back to your wallet. The swap engine is fully functional.

Action 3: Reclaiming Your Assets

A trustworthy exchange must allow users to exit their positions freely. The final test is to burn your LP tokens and withdraw your proportional share of the ETH and TOKEN reserves.

Step 1: Check Your LP Token Balance

Before you can remove liquidity, you need to know how many LP tokens you own. On your Exchange contract, navigate to the Contract tab and then click Read Contract.

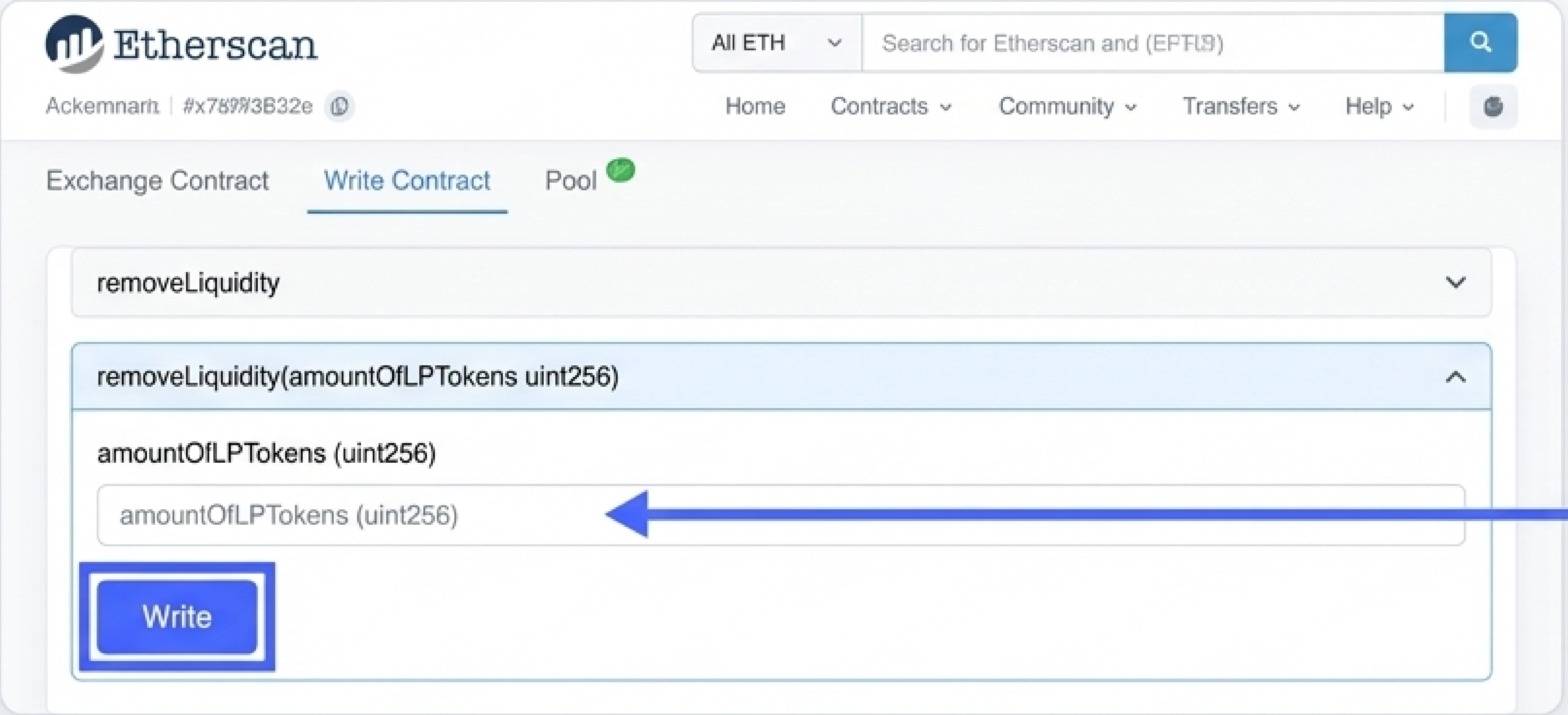


Enter your own wallet address here.

The value returned is your total LP token balance. For the initial liquidity provision of 0.1 ETH, this should be 100000000000000000. Note this value for the next step.

Step 2: Withdrawing Liquidity from the Pool

Go back to the Write Contract tab on your Exchange contract and find the **removeLiquidity** function.

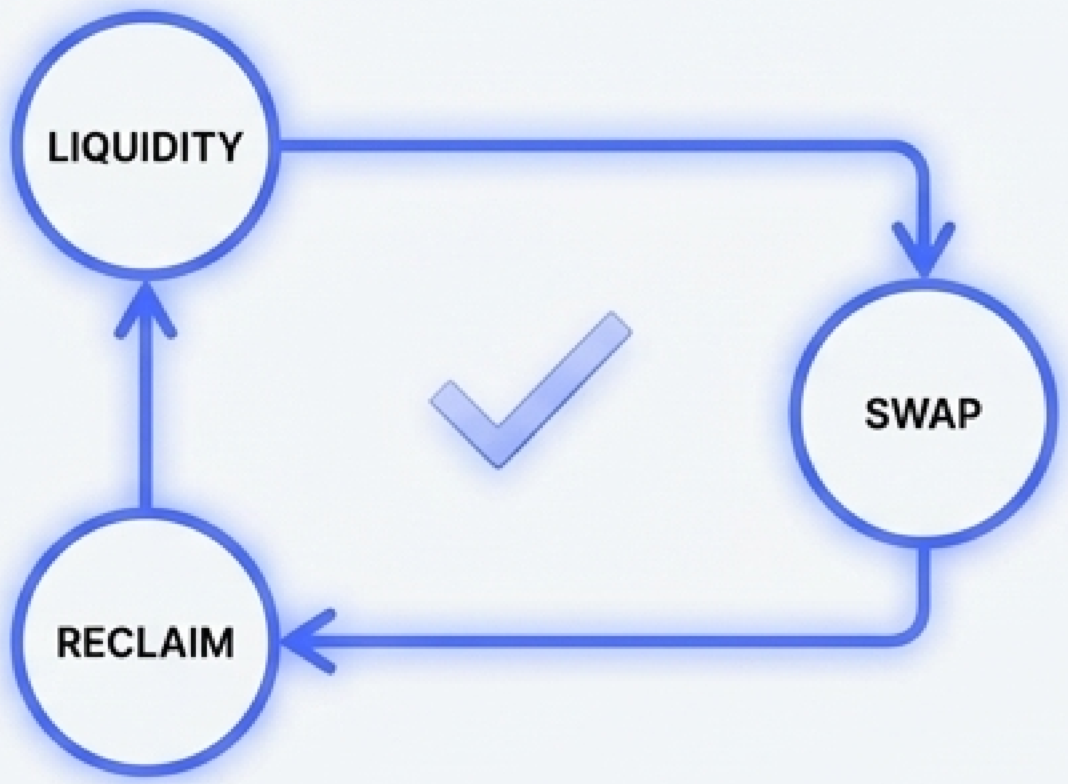


Enter an amount less than or equal to your balance. Let's remove half: 500000000000000000

After the transaction succeeds, you will receive a proportional amount of both ETH and TKN back in your wallet.

Congratulations! You’ve Rebuilt Uniswap v1.

You have now successfully built, deployed, and activated a fully functional decentralized exchange from scratch. You've implemented the core mathematical principles of Automated Market Makers (AMMs) and verified every critical function on a live testnet.



Summary of Achievements

- ✓ Seeded a liquidity pool
- ✓ Executed bidirectional swaps
- ✓ Protected users with slippage parameters
- ✓ Reclaimed assets by burning LP tokens

Have questions or want to celebrate your success? Join the discussion in the Discord Server!